

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	N. Tharun Sai	Reg.No.:	192472382
Department:	CSE AI	Date:	

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
2. Cloud Server Memory Bottleneck
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
3. Autonomous Vehicle Data Processing
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
4. AI Inference Edge Device
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
5. High-Performance Gaming Console
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

*I **N. Tharun Sai (192472382)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

N. Tharun Sai

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

Problem Statement

A smartphone manufacturer observes **lag while switching between multiple applications**. When users switch from one application to another, the processor needs to access application instructions, data, and system resources. If the required data is not available in the faster cache, the processor must access DRAM, which increases latency.

Role of L1, L2, L3 Cache and DRAM

L1 Cache

L1 is the **fastest and smallest cache** available to the processor. It is usually divided into instruction cache and data cache.

- Located very close to the CPU core.
- Provides extremely low access latency.
- Stores frequently used instructions and data.
- Limited capacity because it uses expensive SRAM.

When an application is actively running, frequently used instructions can remain in L1 cache.

L2 Cache

L2 is larger than L1 but slightly slower.

- Provides additional storage when data is not available in L1.
- Usually dedicated to a CPU core or a small group of cores.
- Reduces the number of accesses to L3 and DRAM.

L3 Cache

L3 is larger than L1 and L2 and generally shared by multiple CPU cores.

Its major functions are:

- Store frequently shared data.
- Reduce DRAM accesses.
- Improve communication between CPU cores.
- Improve performance during multitasking.

DRAM

DRAM provides the main working memory of the smartphone.

It stores:

- Running applications
- Operating-system data
- Application data
- Background processes

However, DRAM is much slower than CPU caches.

Proposed Memory Hierarchy

CPU

↓

L1 Cache

↓

L2 Cache

↓

L3 Cache

↓

LPDDR5/LPDDR5X DRAM

↓

UFS Storage

Recommended Improvements

1. Increase L2/L3 cache capacity where power and chip area allow.
2. Use high-speed LPDDR5/LPDDR5X DRAM.
3. Use intelligent cache replacement techniques.
4. Use hardware prefetching to predict required data.
5. Reduce unnecessary background applications.
6. Improve operating-system memory management.
7. Use faster UFS storage for application loading.
8. Keep frequently used application data in RAM.

Expected Result

The improved hierarchy reduces the number of slow DRAM accesses and improves CPU utilization.

Benefits:

- Faster application switching
- Lower response time
- Better multitasking
- Reduced CPU waiting time
- Improved user experience
- Better overall throughput

2. Cloud Server Memory Bottleneck

Problem Statement

A cloud service provider experiences **high latency during database queries**. Thousands of users may simultaneously access databases. If frequently requested data is not available in fast memory, the processor may need to access DRAM or even secondary storage.

Cache Hierarchy

Cache hierarchy provides several levels of fast memory:

CPU → L1 → L2 → L3 → DRAM

Frequently accessed information can remain in cache, reducing access time.

For example, if a database repeatedly accesses the same customer records, indexes, or query results, keeping this information in memory can significantly improve performance.

Virtual Memory Paging

Virtual memory allows a system to use secondary storage as an extension of physical RAM.

Memory is divided into fixed-size **pages**.

If required data is not present in RAM, a **page fault** occurs and the operating system loads the required page from storage.

CPU

↓

RAM

↓

SSD

Paging is useful when physical memory is insufficient, but it is much slower than accessing RAM.

Cache vs Virtual Memory

Parameter	Cache Hierarchy	Virtual Memory
Main purpose	Reduce access latency	Extend available memory
Speed	Very high	Relatively low
Technology	SRAM	DRAM + SSD
Managed by	Hardware	Operating system + hardware
Unit	Cache line	Page
Main problem	Cache miss	Page fault
Database benefit		
Faster repeated access		
Prevents RAM exhaustion		

Recommended Improvements

1. Increase DRAM

Provide sufficient DDR5 memory to keep the active database working set in RAM.

2. Improve CPU Cache

A larger L3 cache can improve access to frequently used data.

3. Database Buffer Pool

Allocate sufficient RAM to the database buffer pool so frequently accessed database pages remain in memory.

4. Use NVMe SSD

If paging or database storage access is unavoidable, NVMe SSD provides much better performance than traditional HDDs.

5. Query Optimization

Database indexes and optimized SQL queries reduce unnecessary memory and storage operations.

6. Caching

Use application-level caching for frequently requested information.

Recommended Hierarchy

CPU



L1 Cache



L2 Cache



L3 Cache



DDR5 DRAM



NVMe SSD

Expected Result

The system can keep frequently accessed data in faster memory while using SSD only when necessary.

This results in:

- Lower database latency
- Fewer page faults
- Higher query throughput
- Better cloud scalability
- Improved response time

3. Autonomous Vehicle Data Processing

Problem Statement

An autonomous vehicle continuously receives huge amounts of data from:

- Cameras
- LiDAR
- Radar
- Ultrasonic sensors
- GPS
- Inertial sensors

This information must be processed in real time. Delayed processing can affect vehicle safety.

Therefore, the memory system must provide **low latency and high bandwidth**.

SRAM

SRAM is extremely fast and does not require periodic refreshing.

Advantages

- Very low latency
- High speed
- Suitable for CPU/GPU/NPU caches
- Useful for real-time processing

Disadvantage

SRAM is expensive and has low storage density.

DRAM

DRAM provides much larger capacity than SRAM.

Advantages

- High capacity
- Lower cost per bit
- Suitable for large sensor datasets
- High bandwidth with modern memory interfaces

Disadvantages

- Higher latency than SRAM
- Requires refresh
- Higher power consumption than some non-volatile technologies

MRAM

MRAM is a **non-volatile memory** technology.

Advantages

- Retains data without power
- Fast access
- Low standby power
- High endurance
- Suitable for critical configuration and persistent data

Disadvantage

Capacity and cost characteristics may not make it a replacement for all DRAM workloads.

Comparison

Feature	SRAM	DRAM	MRAM
Speed	Very high	High	High
Capacity	Low	High	Medium
Volatile	Yes	Yes	No
Latency	Very low	Medium	Low
Power	Higher per bit	Moderate	Low standby
Best use	Cache	Main memory	Persistent data

Proposed Hierarchy

CPU/GPU/NPU



SRAM



High-Bandwidth DRAM



MRAM



Storage

SRAM

Store:

- Current sensor-processing data
- Frequently used AI parameters
- Critical real-time calculations

DRAM

Store:

- Camera frames
- LiDAR point clouds
- Radar data
- AI model working data

MRAM

Store:

- Configuration information
- Calibration data
- Important system parameters
- Selected persistent information

Additional Improvements

- Use multiple DRAM channels.
- Use high-bandwidth memory interfaces.
- Place AI accelerators close to memory.
- Use DMA to transfer sensor data efficiently.
- Use data compression where appropriate.
- Use prefetching for predictable sensor streams.

Expected Result

The proposed hierarchy provides:

Low latency + high bandwidth + adequate capacity + lower standby power

This enables the autonomous vehicle to process sensor information quickly and make decisions in real time.

4. AI Inference Edge Device

Problem Statement

An edge AI device needs to load machine-learning models quickly while consuming minimum power. Edge devices have limited battery capacity and memory compared with cloud servers. For example, an AI camera may need to load an image-classification or object-detection model and process images locally.

NAND Flash

NAND Flash is non-volatile.

It is useful for permanent storage of:

- AI models
- Operating systems
- Applications
- Large datasets

Advantages

- High capacity
- Non-volatile
- Low cost per bit
- Low standby power

Disadvantage

It is much slower than DRAM and SRAM for active computation.

DRAM

DRAM is used as the working memory.

It stores:

- Loaded AI models
- Input images
- Intermediate calculations
- Feature maps

Advantages

- High speed

- High capacity
- Suitable for active AI workloads

Disadvantages

- Volatile
- Higher power consumption than Flash

RRAM

RRAM is a non-volatile memory technology that can provide fast access and low-power operation.

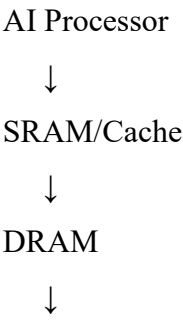
It can potentially be used for:

- AI model weights
- Frequently used parameters
- In-memory computing applications

Comparison

Feature	NAND Flash	DRAM	RRAM
Capacity	High	High	Medium
Speed	Low/Medium	High	High
Power	Low	Higher	Low
Volatile	No	Yes	No
Main use Model storage Active computation Fast persistent model data			

Proposed Hierarchy



RRAM



NAND Flash

Working

1. AI models are permanently stored in NAND Flash.
2. Frequently used model components can be stored in RRAM.
3. The active model is loaded into DRAM.
4. Frequently accessed data is stored in SRAM/cache.
5. The processor performs inference using the fastest available memory.

Additional Optimization

- Quantize AI models from higher precision to lower precision where accuracy permits.
- Compress model weights.
- Use model pruning.
- Load only required model sections.
- Keep frequently used models in faster memory.
- Reduce unnecessary DRAM transfers.

Expected Result

The proposed architecture provides:

- Faster model loading
- Lower power consumption
- Faster inference
- Reduced Flash accesses
- Better battery life
- Improved edge-device performance

5. High-Performance Gaming Console

Problem Statement

A gaming console experiences **frame drops when loading large scenes**. Modern games contain large textures, 3D models, animations, sound files, and lighting information.

When these assets cannot be supplied to the GPU/CPU quickly enough, the processor may wait for data, causing frame drops.

L3 Cache

L3 cache is fast but has limited capacity.

It is useful for:

- Frequently used game-engine instructions
- Physics calculations
- Frequently accessed CPU data
- Shared data between CPU cores

Advantages

- Very low latency
- Reduces DRAM accesses
- Improves CPU performance

Limitation

Its capacity is too small to store complete game scenes.

DRAM

DRAM has much greater capacity and can store large working datasets.

It is used for:

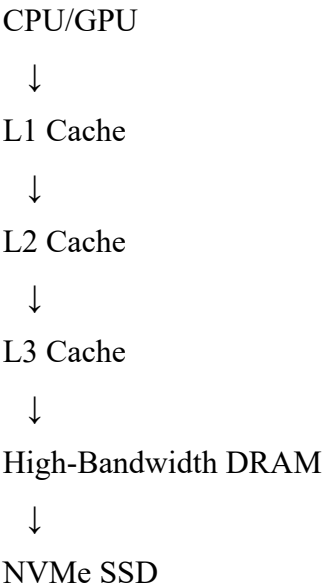
- Textures
- 3D models
- Game assets
- Frame buffers
- AI and physics data

For GPU workloads, high-bandwidth graphics memory is particularly important.

L3 Cache vs DRAM

Feature	L3 Cache	DRAM
Capacity	Small	Large
Latency	Very low	Higher
Cost	High	Lower
Main purpose	Frequently accessed data	Large working datasets
Access	CPU-focused	CPU/GPU system memory
Effect	Reduces latency	Provides capacity and bandwidth

Proposed Hierarchy



Recommended Improvements

1. Increase L3 Cache

A larger L3 cache can keep frequently accessed CPU data closer to the processor.

2. Increase DRAM Bandwidth

High-bandwidth memory allows large amounts of texture and scene data to reach the GPU quickly.

3. Fast NVMe Storage

Use high-speed NVMe SSDs to reduce loading time when assets must be brought from storage.

4. Asset Streaming

Instead of loading the entire game scene at once, load only the assets required for the current area.

5. Prefetching

Predict what assets will be needed next and load them before they are requested.

6. Compression

Compress textures and other assets to reduce the amount of data transferred.

Expected Result

The improved hierarchy allows the console to:

- Load game assets faster
- Reduce CPU/GPU waiting
- Maintain stable frame rates
- Reduce frame drops
- Improve scene transitions
- Increase overall gaming performance

Final Comparison

Application	Main Problem	Important Memory	Recommended Solution
Smartphone	App-switching lag	L1/L2/L3 + DRAM	Larger/faster cache + LPDDR5X
Cloud Server	Database latency	Cache + DRAM	More RAM + caching + NVMe
Autonomous Vehicle	Real-time sensor processing	SRAM + DRAM + MRAM	Low-latency SRAM + high-bandwidth DRAM
Edge AI	Slow model loading/power	NAND + DRAM + RRAM	Hierarchical model storage
Gaming Console	Frame drops	L3 + high-bandwidth DRAM	Larger cache + faster DRAM + NVMe